Time-Series Databases InfluxDB, Prometheus, TimescaleDB

Chapter 06 - Sub-Chapter 07

NoSQL Databases Series

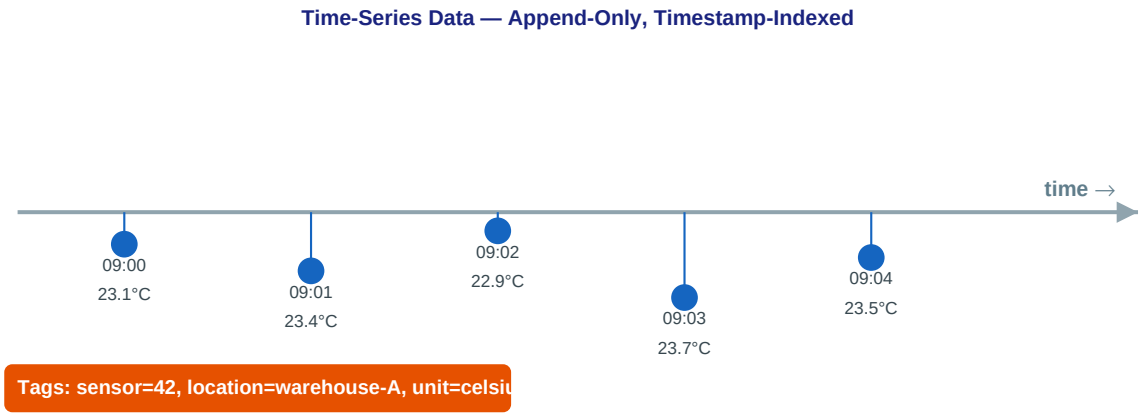
What is a TSDB?	Append-only, timestamp-indexed, columnar
InfluxDB	Flux queries, Java client, downsampling
Prometheus	Pull-based scraping, Spring Boot Actuator
TSDB Landscape & Patterns	TimescaleDB, retention, hot/warm/cold tiers

1 What is a Time-Series Database?

Optimized for data that changes over time

Every metric you track — CPU usage, stock price, heart rate, temperature, API latency — is a time series: a sequence of values ordered by time. This kind of data is special, and general-purpose databases handle it poorly at scale.

A time-series database (TSDB) is purpose-built to store, compress, and query time-stamped data efficiently. The key insight: data always arrives in time order, is almost never updated, and is usually queried as ranges and aggregations — never by individual row lookup.



Why a dedicated TSDB outperforms SQL for metrics:

- **Columnar storage:** Store all timestamps together, all values together — compression ratios of 10-20x vs row-based SQL.
- **Automatic time-based partitioning:** Data is chunked by time (e.g., hourly partitions). Old partitions are cold storage.
- **Downsampling and retention:** Automatically roll up raw data to hourly/daily averages and delete old raw data.
- **Time-aware aggregation:** Built-in functions: windowed averages, rate of change, percentiles, gap-filling.
- **Write throughput:** Millions of data points per second — no index updates on write (timestamps are monotonically increasing).

2 InfluxDB — The Leading Open Source TSDB

Concepts, Flux queries, and Java integration

InfluxDB, created by InfluxData in 2013, is the most widely deployed open-source TSDB. Version 3.x uses Apache Arrow and Parquet for storage, making it extremely fast for both writes and analytical queries.

Core concepts:

- **Bucket** — like a database. Has a retention policy (auto-delete data after N days).
- **Measurement** — like a table name. E.g., 'temperature', 'cpu_usage', 'http_requests'.
- **Tag** — indexed metadata (string key=value). Used to filter/group. E.g., sensor=42, host=web-01.
- **Field** — the actual numeric value. Not indexed. E.g., value=23.7, cpu=85.2.
- **Timestamp** — nanosecond precision. Always UTC.

InfluxDB vs SQL — Same Query, Different Fit

SQL: Avg temperature last hour by sensor

```
SELECT sensor_id,
  AVG(value) AS avg_temp,
  MIN(value) AS min_temp,
  MAX(value) AS max_temp
FROM readings
WHERE ts >= NOW() - INTERVAL '1 hour'
  AND measurement = 'temperature'
GROUP BY sensor_id
-- Works, but slow without partitioning
```

Flux (InfluxDB): Same query, optimized

```
from(bucket: "sensors")
  |> range(start: -1h)
  |> filter(fn: (r) =>
       r._measurement == "temperature")
  |> group(columns: ["sensor_id"])
  |> mean()      // or min(), max()
// Runs on columnar storage -- 100x faster
// for time-range queries
```

Writing data with the Java client:

```
import com.influxdb.client.InfluxDBClient;
import com.influxdb.client.InfluxDBClientFactory;
import com.influxdb.client.WriteApiBlocking;
import com.influxdb.client.domain.WritePrecision;
import com.influxdb.client.write.Point;
import java.time.Instant;

InfluxDBClient client = InfluxDBClientFactory.create(
    "http://localhost:8086",
    "my-super-secret-token".toCharArray()
);

WriteApiBlocking writeApi = client.getWriteApiBlocking();

// Write a single point: measurement, tags, fields, timestamp
Point point = Point.measurement("temperature")
    .addTag("sensor_id",  "sensor-42")
    .addTag("location",   "warehouse-A")
    .addTag("unit",       "celsius")
    .addField("value",    23.7)
    .addField("humidity", 65.2)
    .time(Instant.now(), WritePrecision.NS);

writeApi.writePoint("sensors-bucket", "my-org", point);

// Batch write for high throughput
```

```
List<Point> batch = new ArrayList<>();
for (SensorReading reading : readings) {
    batch.add(Point.measurement("temperature")
        .addTag("sensor_id", reading.getSensorId())
        .addField("value", reading.getValue())
        .time(reading.getTimestamp(), WritePrecision.MS));
}
writeApi.writePoints("sensors-bucket", "my-org", batch);
client.close();
```

Querying with Flux language:

```
import com.influxdb.client.QueryApi;
import com.influxdb.query.FluxRecord;
import com.influxdb.query.FluxTable;
import java.util.List;

QueryApi queryApi = client.getQueryApi();

// Query: average temperature per sensor in the last hour
String flux =
    "from(bucket: \"sensors-bucket\")" +
    "  |> range(start: -1h)" +
    "  |> filter(fn: (r) => r._measurement == \"temperature\")" +
    "  |> filter(fn: (r) => r.location == \"warehouse-A\")" +
    "  |> group(columns: [\"sensor_id\"])" +
    "  |> mean()" +
    "  |> sort(columns: [\"_value\"], desc: true)";

List<FluxTable> tables = queryApi.query(flux, "my-org");
for (FluxTable table : tables) {
    for (FluxRecord record : table.getRecords()) {
        System.out.printf("Sensor: %s, Avg temp: %.2f%n",
            record.getValueByKey("sensor_id"),
            record.getValue());
    }
}
```

Advanced Flux: windowed aggregation and downsampling:

```
// 5-minute windowed averages for the last 24 hours
from(bucket: "sensors-bucket")
  |> range(start: -24h)
  |> filter(fn: (r) => r._measurement == "temperature")
  |> aggregateWindow(every: 5m, fn: mean, createEmpty: false)
  |> yield(name: "5min_avg")

// Rate of change: detect rapid temperature spikes
from(bucket: "sensors-bucket")
  |> range(start: -1h)
  |> filter(fn: (r) => r._measurement == "temperature")
  |> derivative(unit: 1m, nonNegative: false)
  |> filter(fn: (r) => r._value > 2.0)   // spike: more than 2°C/min rise
  |> yield(name: "temperature_spikes")

// Downsampling task (runs automatically, saves storage)
// In InfluxDB UI: Tasks > Create Task
// option task = { name: "downsample-hourly", every: 1h }
// from(bucket: "sensors-bucket")
//   |> range(start: -task.every)
```

```
//   |> aggregateWindow(every: 1h, fn: mean)
//   |> to(bucket: "sensors-hourly")   // write to long-term bucket
```

3 Prometheus — Metrics for Cloud-Native Apps

Pull-based scraping + Spring Boot Actuator

Prometheus is the de facto standard for metrics in Kubernetes environments. Unlike InfluxDB (push), Prometheus pulls (scrapes) metrics from your app's HTTP endpoint. It pairs with Grafana for dashboards and Alertmanager for alerts.

Spring Boot Actuator + Micrometer — zero-config metrics:

```
<!-- pom.xml -- auto-exposes /actuator/prometheus endpoint -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
```

```
# application.yml
management:
  endpoints:
    web:
      exposure:
        include: prometheus, health, info
  metrics:
    export:
      prometheus:
        enabled: true
```

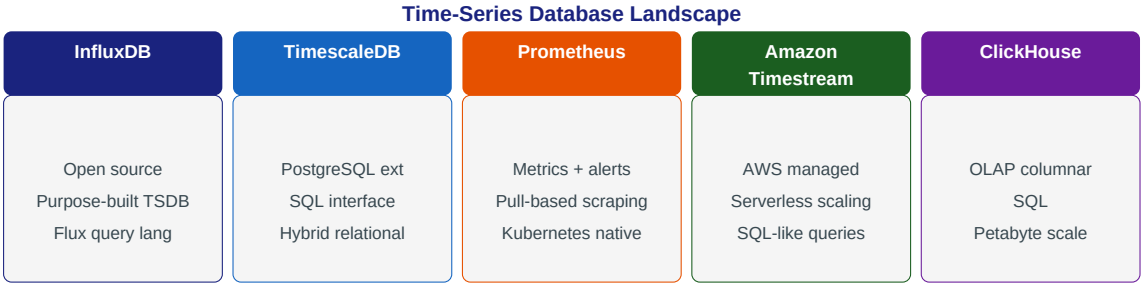
Custom metrics in your service:

```
import io.micrometer.core.instrument.Counter;
import io.micrometer.core.instrument.MeterRegistry;
import io.micrometer.core.instrument.Timer;
import org.springframework.stereotype.Service;
import java.util.concurrent.TimeUnit;

@Service
public class OrderService {

    private final Counter orderCounter;
    private final Timer  orderProcessingTimer;

    public OrderService(MeterRegistry registry) {
        this.orderCounter = Counter.builder("orders.created")
            .tag("channel", "web")
            .description("Total orders created")
            .register(registry);

        this.orderProcessingTimer = Timer.builder("orders.processing.duration")
            .description("Time to process an order")
            .register(registry);
    }

    public Order processOrder(OrderRequest req) {
        return orderProcessingTimer.record(() -> {
            Order order = createOrder(req);
            orderCounter.increment();         // emits: orders_created_total{channel="web"}
```

```
            return order;
        });
    }
}

// Prometheus auto-scrapes http://your-app:8080/actuator/prometheus
// Sample output:
// orders_created_total{channel="web"} 1234.0
// orders_processing_duration_seconds{quantile="0.99"} 0.145
```

4 TSDB Landscape and Patterns

Choosing the right tool, retention, and alerting



TimescaleDB — if you want SQL + time-series:

TimescaleDB extends PostgreSQL with automatic time-based partitioning (hypertables). You write standard SQL, but queries on time ranges are orders of magnitude faster. Perfect if your team knows SQL but needs time-series performance.

```
-- TimescaleDB: create a hypertable (partitioned by time automatically)
SELECT create_hypertable('sensor_readings', 'recorded_at');

-- Insert works exactly like PostgreSQL
INSERT INTO sensor_readings (sensor_id, recorded_at, value)
VALUES ('sensor-42', NOW(), 23.7);

-- Time-range query with downsampling -- uses hypertable chunks (fast!)
SELECT
    time_bucket('5 minutes', recorded_at) AS bucket,
    sensor_id,
    AVG(value) AS avg_temp,
    MAX(value) AS max_temp
FROM sensor_readings
WHERE recorded_at > NOW() - INTERVAL '1 day'
  AND sensor_id = 'sensor-42'
GROUP BY bucket, sensor_id
ORDER BY bucket DESC;

-- Continuous aggregate (auto-materialized view)
CREATE MATERIALIZED VIEW hourly_temp_avg
WITH (timescaledb.continuous) AS
SELECT time_bucket('1 hour', recorded_at) AS hour, sensor_id, AVG(value)
FROM sensor_readings
GROUP BY hour, sensor_id;
```

TSDB patterns: Retention and Hot/Warm/Cold storage:

```
// Pattern: Hot-Warm-Cold data tiering
// Raw data (hot)    : last 7 days  -- full resolution, fast access
// Hourly averages   : last 90 days -- downsampled, moderate access
// Daily averages    : last 2 years -- highly compressed, rarely accessed
// Archive           : forever      -- cold storage (S3/Glacier)

// InfluxDB retention policy
influx bucket create   --name sensors-raw   --retention 7d          # auto-delete data older than
7 days

influx bucket create   --name sensors-hourly   --retention 90d
```

```
// Downsampling task: runs every hour, writes to hourly bucket
from(bucket: "sensors-raw")
  |> range(start: -1h)
  |> aggregateWindow(every: 1h, fn: mean)
  |> to(bucket: "sensors-hourly", org: "my-org")
```

Key Rules

- Tag by dimensions you GROUP BY: sensor_id, host, region, service -- not by value.
- Fields are for measurements (numeric values) -- tags are for metadata (strings).
- Always set a retention policy -- raw IoT data at 1-second granularity grows fast.
- Use Prometheus for application metrics (latency, error rate, throughput) in Kubernetes.
- Use InfluxDB or TimescaleDB for IoT sensor data, financial tick data, server monitoring.